Новые ключи setting.txt

Справочник по настройке движка Quantum (J2ME, MIDP-2.0 / CLDC-1.1) под аппараты класса ARM9. Ветка arena/50252a64-quantum, коммит 7504862, дата 24.10.2026.

Коротко о совместимости. Формат setting.txt не изменился: те же строки КЛЮЧ=ЗНАЧЕНИЕ, те же комментарии # и ;. Все перечисленные ниже ключи **опциональны** - с оригинальным setting.txt движок ведёт себя как раньше, потому что на «быстром» профиле значения по умолчанию совпадают с прежним поведением. Форматы уровней, мешей, текстур, лайтмапов (lightmap.vla) и записей RMS не менялись: новые ключи влияют только на то, какая ветка кода выполняется.

1. Как читается setting.txt

Файл читается один раз при старте: Main.init() создаёт GameIni.createGameIniFromResource("/setting.txt", true) и сразу передаёт его в DeviceProfile.init(settings) - раньше defaultSettings() и раньше чтения сохранённого профиля из RMS, чтобы setPersQ() уже знал свой шаг. Разбор выполняет IniFile (модуль UtilsText): строки режутся по \n, пустые пропускаются, строки, начинающиеся с # или ;, считаются комментарием, остальные разрезаются по первому =, ключ и значение.trim() кладутся в хеш-таблицу.

Правило	Подробность
Куда писать ключи	setting.txt читается с поддержкой групп, поэтому новые ключи надо писать **в корне файла - до первой строки вида** [группа]. DeviceProfile читает их через getNoLang()/getIntNoLang(), то есть из корневой таблицы.
Регистр и пробелы	Имя ключа чувствительно к регистру (пишите заглавными, как в оригинале). Пробелы вокруг = игнорируются: ключ и значение.trim()аются. Значение CPU_PROFILE регистр не важен (AUTO/auto/Auto равнозначны).
Комментарии	**Только отдельной строкой.** Комментарий в конце строки не поддерживается: в FAST_LIGHT=1 ; включить значением станет строка 1 ; включить.
Тип значения	Все числовые ключи разбираются через Integer.parseInt() без перехвата исключений, поэтому значение обязано быть целым числом. Нечисловое значение - это NumberFormatException при старте (см. предупреждение ниже).
Неизвестные ключи	Игнорируются: таблица просто не спрашивается о них. Старые ключи оригинала продолжают работать как работали.
Отсутствие ключа	Берётся значение по умолчанию из профиля устройства (колонки «ARM9» и «FAST/AUTO» в таблице ниже). Отсутствие файла или ключа - не ошибка.
Приоритет	Для ключей уровня (UPDATE_ONLY_NEAR и подобные): level.ini важнее setting.txt, setting.txt важнее профиля устройства, профиль важнее встроенного значения. Явный ключ в файле всегда перекрывает профиль.
Когда применяется	Один раз при старте. Смена значения требует перезапуска MIDlet.

Осторожно: нечисловое значение роняет старт. DeviceProfile.init() не перехватывает NumberFormatException, поэтому FAST_LIGHT=on, AI_LOD=1 ; включить или FAST_CULL=yes приведут к исключению в Main.init(). Допустимы только целые числа (для CPU_PROFILE - слова AUTO/ARM9/FAST или цифры 1/2).

Рабочий пример - все новые ключи сразу (значения профиля ARM9):

```
; --- Quantum: настройка под ARM9 (все ключи опциональны) ---
CPU_PROFILE=AUTO
FAST_LIGHT=1
PERSPECTIVE_STEP=32
NEAR_ONLY=1
AI_LOD=1
AI_LOD_DIST=10000
MORPH_LOD=1
MORPH_LOD_DIST=12000
FAST_CULL=1
```

2. Сводная таблица ключей

Тринадцать новых ключей. «ARM9» и «FAST/AUTO» - значения по умолчанию на соответствующем профиле; явный ключ в файле перекрывает их в обоих случаях.

Ключ	Значения	ARM9	FAST / AUTO	Что делает
`CPU_PROFILE`	AUTO | ARM9 | FAST (или 1 | 2)	**AUTO**	**AUTO**	принудительный выбор профиля устройства
`FAST_LIGHT`	0 | 1	**1**	**0**	целочисленное освещение вместо double
`PERSPECTIVE_STEP`	0 ... 256	**32 (если ключ не задан)**	**0**	шаг перспективной коррекции (Main.q)
`NEAR_ONLY`	0 | 1	**1**	**0**	симулировать только объекты рядом с игроком
`AI_LOD`	0 | 1	**1**	**0**	далёким ботам реже искать цель
`AI_LOD_DIST`	единицы карты (по умолчанию 10000)	**10000**	**10000**	порог «далёкости» для AI_LOD
`MORPH_LOD`	0 | 1	**1**	**0**	дальним мешам не смешивать кадры анимации
`MORPH_LOD_DIST`	единицы карты (по умолчанию 12000)	**12000**	**12000**	порог «далёкости» для MORPH_LOD
`FAST_CULL`	0 | 1	**1**	**0**	быстрый отсек BoundingBox перед полным тестом
`QCT`	0 | 1	**1**	**1**	подхватывать упакованные текстуры .qct рядом с .png
`FAST_SETUP`	0 | 1	**1**	**0**	подготовка перспективных полигонов без машинных делений (коэффициенты тумана)
`RENDER_SCALE`	25...100	**100**	**100**	внутреннее разрешение рендера в процентах от экрана; flush() растягивает кадр nearest-ом
`NOKIA_BLIT`	0 | 1	**1**	**1**	вывод кадра через Nokia DirectGraphics.drawPixels() вместо g.drawRGB()

3. Профиль устройства: что включает ARM9

Профиль определяется один раз и хранится в `DeviceProfile.cpu` (`CPU_AUTO` = 0, `CPU_ARM9` = 1, `CPU_FAST` = 2); флаг `lowEnd = (cpu == CPU_ARM9)` и задаёт значения по умолчанию для пяти ключей: FAST_LIGHT, NEAR_ONLY, AI_LOD, MORPH_LOD, FAST_CULL (при lowEnd - 1). На постэффекты профиль не влияет: ими управляют родные ключи игры BLOOM, VIGNETTE (DOUBLE_BRIGHT вырезан). PERSPECTIVE_STEP на ARM9 получает 32, если ключа нет в файле.

Профиль	Когда выбирается	Что это значит
FAST	Десктоп, эмулятор, Symbian/S60, UIQ; явно - CPU_PROFILE=FAST или =2	Все значения по умолчанию прежние: движок ведёт себя как оригинал. Работают только точные ускорения (раздел 6).
ARM9	Семейства J2ME-аппаратов без FPU и аппаратного делителя (Sony Ericsson DB20xx, Samsung, Motorola, LG, Siemens, Philips, Nokia Series40 и др.), любая платформа с CLDC-1.0; явно - CPU_PROFILE=ARM9 или =1	Включаются щадящие значения по умолчанию: целочисленное освещение, без постэффектов, крупнее шаг перспективной коррекции, обновляются только ближайшие объекты, LOD для ИИ и морфинга, быстрый отсек боксов.
AUTO	Платформа не распознана (значение по умолчанию)	lowEnd = false, то есть поведение как на FAST: включены только точные ускорения. Это страховка - на неизвестном аппарате ничего не ухудшается.

4. Подробно по каждому ключу

CPU_PROFILE

Значения: AUTO | ARM9 | FAST (или 1 | 2) | по умолчанию: ARM9 - AUTO, FAST/AUTO - AUTO

Где применяется. DeviceProfile.init(); вызывается один раз из Main.init() - до defaultSettings() и до чтения сохранённого профиля из RMS.

Что ускоряет. Не ускорение само по себе, а переключатель: профиль ARM9 включает щадящие значения по умолчанию для всех остальных ключей, профиль FAST/AUTO оставляет прежнее поведение.

Что теряется. Нет - это выбор режима.

Подробности

- Регистр значения не важен; вместо слов принимаются 1 (= ARM9) и 2 (= FAST). Любое другое значение (включая 0 и пустое) трактуется как AUTO.
- AUTO означает, что платформа не распознана: тогда lowEnd = false и работают только точные ускорения (те, что не имеют ключа вовсе).
- Автоопределение FAST: десктоп и эмуляторы (os.name содержит win/mac/nix/nux/aix/android), Symbian/S60 (список из 54 моделей Nokia), pstros, KEmulator, FreeJ2ME, J2ME Loader, UIQ-аппараты Sony Ericsson (P*, M600, W950, G700, Xperia*).
- Автоопределение ARM9: SonyEricsson*/SEMCCP*/SEMC*, Samsung/SAMSUNG, Motorola/MOT-/MOTO, LG-/LG//LGE, Fly, Philips, SAGEM/Sagem, BenQ, SIEMENS/Siemens, Panasonic, SHARP, Toshiba, Alcatel, ZTE, Huawei, Nexian, остальные Nokia* (Series40), а также любая платформа с microedition.configuration = CLDC-1.0.
- System.getProperty() обёрнут в try/catch(Throwable): на части аппаратов он бросает исключение для неизвестных ключей, и старт не должен из-за этого падать.

FAST_LIGHT

Значения: 0 | 1 | по умолчанию: ARM9 - 1, FAST/AUTO - 0

Где применяется. MathUtils.calcLightFast() / calcLightDirFast() / microCalcLightFast(); выбор ветки - в RenderObject.microLight(), RenderObject.dirLight() и MathUtils.calcLightRender() (drawmode 5, 6 и 12; всего 18 мест в Polygon3V/4V, LightedPolygon3V/4V, ColorLightedPolygon3V/4V, TexturingAffine, TexturingPers).

Что ускоряет. Убирает из кадра double-арифметику, Math.sqrt и лишние деления: на ARM9 нет FPU, поэтому float/double и Math.sqrt выполняются программно (сотни тактов на вызов), а деление - библиотечная подпрограмма.

Что теряется. Освещение может отличаться на 1-4 единицы из 255 (см. точность ниже). На глаз в реальной сцене не различимо.

Подробности

- Точность проверена перебором (docs/verify/test_light.py): calcLightFast - 98,1 % значений совпадают бит-в-бит, худшее отклонение 3 из 255; calcLightDirFast - 96,7 % и 4 из 255. В реальной сцене (lightdir порядка 4096) отклонение 0-1 из 255.

- Все 8 граничных случаев (нулевые векторы, оси, (4096,0,0) против (-4096,0,0), 30000*6, 32767*3 против (1,1,1)) совпадают точно.
- Оригинальные MathUtils.calcLight() и microCalcLight() НЕ изменены: ими пользуется LightMapper, который печёт лайтмапы и может писать файлы, - там прежняя математика нужна бит-в-бит. Поэтому формат и содержимое lightmap.vla не меняются.
- Переполнение суммы квадратов в оригинале даёт NaN, а (int)NaN == 0: быстрая ветка возвращает 0 в том же случае. Clamp сохранён (calcLightFast - clamp255, microCalcLightFast - только верхний 255, потому что оригинал может вернуть отрицательное значение и это используется).

PERSPECTIVE_STEP

Значения: 0 ... 256 | по умолчанию: ARM9 - 32 (если ключ не задан), FAST/AUTO - 0

Где применяется. Main.setPersQ(): q = PERSPECTIVE_STEP, если он больше 0, иначе 24. Применяется только при persQ меньше либо равно 2 (пункт меню настроек «перспектива»); при persQ = 3 и 4 q всегда 12.

Что ускоряет. q - длина сегмента в пикселах, который растрируется одним набором инкрементов. Чем крупнее шаг, тем реже выполняется дорогое 64-битное деление в перспективном растеризаторе.

Что теряется. Небольшое искажение текстуры на длинных наклонных сегментах (пол/потолок вдали от камеры): коррекция выполняется реже.

Подробности

- 0 означает «как раньше» (q = 24) - значение по умолчанию на профилях FAST и AUTO.
- На профиле ARM9, если ключ в файле отсутствует, ставится 32. Явный ключ всегда важнее профиля: PERSPECTIVE_STEP=0 на ARM9 вернёт 24.
- Отрицательные значения обрезаются до 0, большие 256 - до 256.

NEAR_ONLY

Значения: 0 | 1 | по умолчанию: ARM9 - 1, FAST/AUTO - 0

Где применяется. LevelLoader: Main.updateOnlyNear = getBoolean("UPDATE_ONLY_NEAR", lvl, setting, DeviceProfile.nearOnly); затем Scene (5 мест) и NPCSpawner используют house.getNearObjectsForUpdate() / getNearParts() вместо полного списка объектов.

Что ускоряет. Обход и обновление всех объектов сцены - одна из главных статей расходов в кадре с NPC: вместо всей карты обрабатывается только текущая часть (part) и соседние.

Что теряется. Объекты вдали от игрока не обновляются: их ИИ и физика «застывают» до приближения. NPCSpawner при этом проверяет дальние точки раз в 8 секунд, поэтому возрождение не ломается.

Подробности

- Ключ не вводит новое поведение - он задаёт значение по умолчанию для существующих ключей оригинала UPDATE_ONLY_NEAR и UPDATE_ONLY_NEAR_PHYSICS (level.ini или setting.txt). Ключ уровня и ключ в setting.txt по-прежнему важнее профиля.
- UPDATE_ONLY_NEAR_PHYSICS по умолчанию наследует итоговое значение UPDATE_ONLY_NEAR, как и в оригинале.

AI_LOD

Значения: 0 | 1 | по умолчанию: ARM9 - 1, FAST/AUTO - 0

Где применяется. Bot.update() раз в кадр ставит флаг aiFar (DeviceProfile.aiLod && distance > aiLodDist); NPC.action() использует react = aiFar ? (reacttimer меньше на один разряд: reacttimer << 1) : reacttimer.

Что ускоряет. Блок «поиск цели и перестроение маршрута» тянет findBot() по всем объектам сцены и проверку видимости. Для дальнего бота он выполняется вдвое реже. Флаг считается один раз на бота в кадр, потому что action(Scene) игрока не видит, а scene.findPlayer() - это обход всех объектов.

Что теряется. Далёкий NPC замечает игрока и перестраивает маршрут на один «такт реакции» позже. Атака, движение, анимация и физика работают с прежней частотой.

Подробности

- Порог «далёкости» задаётся ключом AI_LOD_DIST.

AI_LOD_DIST

Значения: единицы карты (по умолчанию 10000) | по умолчанию: ARM9 - 10000, FAST/AUTO - 10000
Где применяется. Bot.update(): character.distance(player.character) > DeviceProfile.aiLodDist.

Что ускоряет. Позволяет выбрать, какую долю ботов считать «дальними». Внутри хранится квадрат порога (10000 * 10000 = 100 000 000), поэтому сравнение идёт без извлечения корня.

Что теряется. Чем меньше значение, тем больше ботов переходят в редкий режим и тем заметнее задержка их реакции.

Подробности

- Значения больше 46000 обрезаются до 46000: 46000 в квадрате = 2 116 000 000 - последнее значение, которое помещается в int без переполнения.
- Значения меньше либо равны 0 игнорируются - остаётся 10000.
- Ключ не зависит от профиля CPU и имеет смысл только при AI_LOD=1.

MORPH_LOD

Значения: 0 | 1 | по умолчанию: ARM9 - 1, FAST/AUTO - 0
Где применяется. MeshImage.morphLod() и setAnimationLod(): на время mesh.setAnimation() временно выключается animation.morphEnabled и сразу восстанавливается.

Что ускоряет. Morphing.interpolation() считает 6 умножений и 6 сдвигов на КАЖДУЮ вершину каждый кадр. Дальний меш занимает на экране несколько пикселов, поэтому смешивание кадров для него бессмысленно.

Что теряется. У далёких персонажей анимация переключается целыми кадрами, без плавного смешивания. На дистанции дальше MORPH_LOD_DIST это практически не заметно.

Подробности

- Флаг morphEnabled принадлежит анимации, а анимация может быть общей для нескольких NPC (кэш Asset), поэтому он восстанавливается сразу после вызова - на остальные меши отключение не влияет.

MORPH_LOD_DIST

Значения: единицы карты (по умолчанию 12000) | по умолчанию: ARM9 - 12000, FAST/AUTO - 12000
Где применяется. MeshImage.morphLod(): sz < -(DeviceProfile.morphLodDist меньше на два разряда: << 2), где sz = centre.rz * 4.

Что ускоряет. Задаёт, с какой дистанции меш считается дальним. Поскольку перед камерой rz отрицателен, условие эквивалентно «дистанция больше MORPH_LOD_DIST».

Что теряется. Чем меньше значение, тем больше мешей теряют плавность анимации.

Подробности

- Отрицательные значения обрезаются до 0, значения больше 1 000 000 - до 1 000 000, чтобы сдвиг на два разряда не переполнил int.
- Ключ не зависит от профиля CPU и имеет смысл только при MORPH_LOD=1.

FAST_CULL

Значения: 0 | 1 | по умолчанию: ARM9 - 1, FAST/AUTO - 0
Где применяется. BoundingBox.isVisible(): if (DeviceProfile.fastCull && !maybeVisible(g3d, matrix, x1, y1, x2, y2)) return false; вызовы isVisible - NPC (2), BigZombie, Zombie, Corpse, MeshObject, RoomObject (6), House, Room.

Что ускоряет. Полный тест - это 8 преобразований вершин и 16 целочисленных делений на каждый объект каждый кадр (idiv на ARM9 - библиотечная подпрограмма). Предварительная проверка центра с радиусом стоит одно преобразование и два деления: около 30 дешёвых целочисленных операций.

Что теряется. Визуально не теряется ничего: быстрый тест отбрасывает только заведомо невидимое. На видимых объектах появляются накладные расходы около 15 % от стоимости

проверки видимости (сам рендер меша на порядки дороже).

Подробности

- Все оценки направлены «в сторону не отсекать»: радиус бокса умножается на наибольшее по модулю значение строки матрицы, а знаменатель проекции берётся для ближайшей к камере точки (максимальное увеличение).
- Быстрый тест возвращает false только тогда, когда полный тест тоже вернул бы false. Проверено перебором: docs/verify/test_cull.py - 182 187 случаев, 0 ошибочных отсечений. На 60 000 реалистичных случаев точный тест отвергает 92,5 % объектов, быстрый - самостоятельно 48,3 % (остальные добирает полный тест).
- При любом выходе за безопасные целочисленные границы (вырожденный или перевёрнутый бокс, пустая/перевёрнутая область отрисовки, слишком большое расстояние, cullOk = false) объект отправляется на полный тест.

QCT

Значения: 0 | 1 | по умолчанию: ARM9 - 1, FAST/AUTO - 1

Где применяется. RawImage.createRawImage(): для пути с .png движок ищет в JAR одноимённый ресурс .qct и, если он есть, читает его родным загрузчиком qctToRawImage(); иначе - прежний PNG-путь.

Что ускоряет. Меньше вес текстур и память на загрузке: QCT - 0,75 байта на пиксель плюс палитра против 4 байт на пиксель в int[]; декодированию не нужен промежуточный Image (PNG-путь держит Image и int[] одновременно).

Что теряется. В рантайме ничего: точность цвета фиксируется при упаковке (палитра 2-64 цвета; на градиентах с малой палитрой возможен бендинг). Скорость рендера не меняется - после распаковки текстура живёт в том же int[].

Подробности

- Формат родной для движка: загрузчик qctToRawImage существовал всегда, не хватало энкодера. Теперь он есть - code.utils.QctEncoder (DXT1-подобные блоки 2x2 с глобальной палитрой, целочисленный k-means, run-поток полутонов альфы), а конвертер PNG -> QCT собран отдельным MIDlet QctConverter.jar (docs/j2me-build/build-qctconv.sh, описание в docs/ARM9.md §10).
- Ограничение формата: ширина и высота чётные (блоки 2x2); нечётные текстуры энкодер отклоняет, а движок и раньше читал бы их с потерей последней строки/колонки.
- Ключ не зависит от профиля CPU: это формат хранения, а не ухудшение. QCT=0 возвращает чисто png-поведение; без .qct в JAR поведение не меняется вовсе.
- Проверено: docs/verify/test_qct.py - round-trip точен на палитрных текстурах и бинарной альфе, полутона альфы сохраняются, а настоящий загрузчик движка даёт побитово тот же int[], что и QctEncoder.decode().

FAST_SETUP

Значения: 0 | 1 | по умолчанию: ARM9 - 1, FAST/AUTO - 0

Где применяется. TexturingPers.paint9Pers(): fogCoef/waterCoef заменяют 0xff + rz*0xFF/fDist и /waterDistance умножением на floor(2^34/d); магическое число кэшируется и пересчитывается только при смене делителя. При d < 256 сохраняется прежний idiv.

Что ускоряет. Из примерно 23 idiv на полигон в setup исчезают 6 туманных; остальное делает не зависящая от ключа точная часть: 14 градиентов ((delta<<12)/dy) заменяются одним long-делением на полигон и 14 умножениями, три /255 - тождеством без деления. В сценах с сотнями полигонов это миллионы тактов в секунду.

Что теряется. Градации тумана и подводной дымки могут отличаться на 1 из 255 (94 % значений совпадают точно). На глаз неотличимо; видно только при покадровом сравнении скриншотов.

Подробности

- Точные замены этого этапа ключа не имеют вовсе: div255 (тождество (a+1+(a>>8))>>8, guard |x| < 65535, вне диапазона - прежний idiv) и градиенты (ceil(2^37/dy) с однократной коррекцией перелёта - точно для всех int при dy >= 64, при dy < 64 - прежний idiv).

- Погода (WeatherGenerator) хранит координаты частиц в int Q8 (1/256 пиксела) и не делает делений на частицу вовсе: /50 через FastMath.div50 (бит-в-бит), деление на глубину 1..65 - умножением на таблицу INV_DEPTH. Ключа нет: отличие от float субпиксельное (docs/ARM9.md §11.3).
- Деления wz = templ/(-rz+move) (3 на полигон) оставлены: делитель разный для каждой вершины, кэш магических чисел не помогает.
- Проверено: docs/verify/test_setup_math.py - 0 расхождений div255, div50 и градиентов (критические точки k*d±1, отрицательные, случайные по всему int), максимальная погрешность тумана 1; TestWeather - инварианты 400 кадров движения частиц.

RENDER_SCALE

Значения: 25…100 | по умолчанию: ARM9 - 100, FAST/AUTO - 100

Где применяется. DirectX7.resize(): view = target*S/100 (не меньше 1). flush(): страж renderScale >= 100 - прежний drawRGB(display) бит-в-бит; иначе nearest-апскейл в scaleBuf через colMap/rowMap (строятся один раз на resize, пиксельный цикл - чистая копия). HUD, оружие, прицел, леттербокс, PauseScreen, диалоги и фон - на target-размерах; растеризаторы и постэффекты - на view-буфере.

Что ускоряет. Растеризаторы платят за каждый пиксел внутреннего буфера (§6, §18): при RENDER_SCALE=75 пикселов 56 %, при 50 - 25 %. На аппаратах, где узкое место fillrate, это самый сильный рычаг после displaySize; в отличие от displaySize, HUD, прицел и тексты остаются в родном разрешении.

Что теряется. 3D-картинка становится крупнее зерном: nearest-апскейл даёт «блоки» размером около 100/S пикселов. HUD, прицел, сообщения и фон паузы не затрагиваются - они рисуются в целевом разрешении.

Подробности

- По умолчанию 100 у любого профиля (opt-in): без ключа рендер бит-в-бит прежний, единственная цена ключа при 100 - одно int-сравнение во flush().
- Значения ниже 25 обрезаются до 25, выше 100 - до 100. Ключ складывается с displaySize: сначала displaySize задаёт целевой размер леттербокса, затем RENDER_SCALE - внутренний.
- Погода учтена: частицы дождя живут в экранном пространстве (рисуются на Graphics), а вспышка молнии пишет градиент в view-буфер его же страйдом; страйд getRGB виньетки исправлен на img.getWidth() (docs/ARM9.md §22.3).
- Проверено: docs/verify/test_render_scale.py - зеркала resize/парсинга/карт/апскейла (границы, сюръективность, бит-в-бит при S=100), границы lightingEffect и виньетки, инварианты исходников.

NOKIA_BLIT

Значения: 0 | 1 | по умолчанию: ARM9 - 1, FAST/AUTO - 1

Где применяется. DirectX7.flush(): детектор nokia(Graphics) один раз выполняет Class.forName("com.nokia.mid.ui.DirectGraphics"); при успехе кадр уходит в NokiaBlitter.blit() = drawPixels(pixels, alpha, 0, scanlength, x, y, w, h, 0, TYPE_INT_8888_ARGB). На аппаратах без Nokia UI класс NokiaBlitter не загружается вовсе - автоматический откат на g.drawRGB(). Совместим с RENDER_SCALE (обе ветви масштаба).

Что ускоряет. На Nokia S60/S40 drawPixels отдаёт пикселы нативному графическому ядру, минуя промежуточное копирование MIDP-реализации: единственный вывод кадра за кадр становится дешевле. Выигрыш зависит от прошивки (порядка 5-15 % времени вывода кадра).

Что теряется. Не теряется ничего: адресация и смысл transparency у drawPixels совпадают с drawRGB пиксел-в-пиксел (официальный javadoc Nokia UI API 1.1). На аппаратах без com.nokia.mid.ui ключ не имеет значения - откат автоматический.

Подробности

- NOKIA_BLIT=0 - страховка: возвращает ровно прежний путь вывода даже на Nokia (например, если на конкретной прошивке drawPixels окажется медленнее drawRGB).

- Компиляция идёт против стаба com.nokia.mid.ui (docs/j2me-build/stubsrc-nokia) - в JAR он не попадает; значения констант (TYPE_INT_8888_ARGB=8888 и др.) сверены с официальным javadoc и двумя независимыми эмуляторами, так как ECJ инлайнит static final int.
- Проверено: docs/verify/test_nokia_blit.py - зеркала детектора и парсинга ключа, адресация drawPixels==drawRGB, константы стаба, инварианты исходников и содержимого JAR (docs/ARM9.md §24).

5. Ключи оригинала, у которых изменилось значение по умолчанию

Эти ключи существовали раньше и не переименованы. Изменилось только то значение, которое подставляется, когда ключ не задан ни в level.ini, ни в setting.txt: теперь оно берётся из профиля устройства (ключ NEAR_ONLY).

Ключ	Где задаётся	Раньше по умолчанию	Теперь по умолчанию	Смысл
`UPDATE_ONLY_NEAR`	level.ini или setting.txt	`false`	DeviceProfile.nearOnly (то есть 1 на ARM9, 0 на FAST/AUTO)	Симулировать и сталкивать только объекты рядом с игроком. Ключ существует в оригинале; изменилось лишь значение, которое подставляется, если ключ нигде не задан.
`UPDATE_ONLY_NEAR_PHYSICS`	level.ini или setting.txt	значение `UPDATE_ONLY_NEAR`	без изменений (наследует UPDATE_ONLY_NEAR)	Отдельное управление физикой ближайших объектов. Поведение то же, но наследуемое значение теперь может прийти из профиля.

Если в level.ini или setting.txt оригинала эти ключи уже прописаны, их значение важнее - профиль не подменяет явно заданное. Поэтому уровни, которые рассчитывали на полное обновление объектов, продолжают работать так же.

6. Ускорения без ключа: включены всегда

У этих изменений ключа нет вовсе - они либо бит-в-бит эквивалентны оригиналу, либо их погрешность ограничена и проверена перебором. Отключать их нечем и незачем: форматы данных и результаты они не меняют.

Что сделано	Где	Точность
FrameClock - один System.currentTimeMillis() на кадр	QFPS.frame() кэширует время; 12 читателей: Sprite.updateFrame, Texture.updateAnimation, NPC.render/setSprite/action/isTimeToRenew, Player (интервал шага 450 мс, проверка Y 1000 мс, таймер смерти 3000 мс), GameScreen (подсветка оружия 135 мс, оверлей, сообщение)	расхождение не больше длительности кадра; событийные записи таймеров (lastAttack и подобные), ограничитель кадров в GameScreen и измерение времени загрузки остались на настоящем времени
FastMath.div100() вместо деления на 100	индекс кадра анимации: NPC.render, Zombie.render, BigZombie.render, GameObject.getFrameInterDiv()	бит-в-бит: умножение на 1 374 389 535 и сдвиг на 37, знак обрабатывается отдельно; проверено на 6 312 931 значении, 0 расхождений (docs/verify/test_fastmath_java.py)
Vector3D.length() через FastMath.isqrt()	все вызовы length() в физике, ИИ и рендере	бит-в-бит: isqrt(n) == (int)Math.sqrt(n) для любого int n
FastMath.div2() вместо /2	Sprite.project() (обе версии), NPC.render (muzzleFlash.getHeight() / 2)	бит-в-бит, включая отрицательные значения (>> 1 для них не эквивалентен)
32-битные остатки в анимации вместо 64-битных	Sprite.updateFrame(), Texture.updateAnimation()	Sprite - бит-в-бит (время неотрицательно и помещается в int); Texture - фаза отсчитывается от первого кадра, период, скорость и взаимная синхронность текстур не меняются

Что сделано	Где	Точность
Физика и AI (NPC): кэш AABB, фильтры текстур, кэши списков	кэш AABB полигонов Mesh для статической геометрии комнат (инвалидация в transformSave); фильтр неколлизионных текстур поднят до дорогих тестов (underRoof сохранён); XZ-отсечка в isPointOnMesh; список объектов кэшируется на кадр физики (Room.objectsRevision); ближние комнаты Room[] вместо перестройки Vector; findBot через copyInto	fuzz старых и новых версий sphereCast/computeHeight/isPointOnMesh: 360 тыс. сравнений бит-в-бит, включая мутации позиции сферы (docs/verify/test_ai_physics.py)
Фикс мерцания [MESH]-объектов (§20): устойчивая сортировка + reSort без мутации вершин	fallback сортировки буфера полигонов - устойчивые слияния (порядок равных sz всегда равен порядку добавления и не меняется между кадрами); BoundingBox.reSort() считает трансформированный AABB напрямую из minx..maxz и не вызывает transformSave(): та мутировала вершины, и бокс каждый кадр накапливал умножение на матрицу - полный тест видимости [MESH]-объектов возвращал случайные результаты (мерцание/пропадание)	test_sort.py (65 случаев + 50 кадров дрейфа, сравнение с устойчивым эталоном; свидетель: старый гибрид ломал порядок равных на 11 кейсах), test_bbox.py (9 боксов x 3 серии матриц x 60 кадров: 0 расхождений с оригиналом; свидетель: старый код - 335 флипов isVisible)
Деления /wz в ядрах полов и перспективы: 64 бита -> 32 бита и обратное умножение (§21)	TexturingFloors.divPairFp(): 39 мест (9 функций полов + 10 ядер перспективы) - путь A бит-в-бит (32-битные деления), путь B через общее обратное rq=2^30/wz + умножения (погрешность <= 65 fp-единиц = 1/63 тексела, класс FPq, не накапливается), fallback - прежние точные 64-битные	fuzz 300 612 пар: A и fallback бит-в-бит, B <= 64 fp; 120 симуляций пролёта - отклонение траектории <= 59 fp (docs/verify/test_wz_div.py)
Режимы 9/13 и ядра перспективы: FPq, кэш fDist/255, div3, div255	сегментное /q -> *FPq>>fp в пяти ядрах (_1/_3/_3_HQ/_5/_6_HQ, как уже было в _13); fDist/255 через ключ-кэш; (af+ag+ab)/3 и ar*light/255 через FastMath.div3/div255	div3/div255 бит-в-бит (540 тыс. значений перебором); FPq - отклонение <= 0,04 тексела на шаг, не накапливается (docs/verify/test_modes913.py)
Стекло (режим 7): преломление через байтовые таблицы	три умножения на пиксель в paint_glass/paintPers_glass -> две выборки из int[256], таблицы кешируются на размер экрана	бит-в-бит: 120 864 случая - все R,G 0..255 x scale 1..256 и реальные экраны (docs/verify/test_glass.py)
Room.divChunk() / divChunkRender()	18 мест деления в растеризации комнат	бит-в-бит: 3020 пар значений, 0 расхождений (docs/verify/test_room_div.py)
Бюджетный гибрид сортировки по глубине	сортировка полигонов в кадре; fallback - устойчивые слияния (§20: порядок равных sz постоянен, компланарные [MESH] не мерцают)	бит-в-бит + устойчивость: 65 случаев и 50 кадров дрейфа, 0 расхождений (docs/verify/test_sort.py)
RenderObject.kind вместо instanceof	13 мест в рендере и сортировке	бит-в-бит: поле ставится в конструкторе после super()
House.getNearObjectsCached()	обновление ближайших объектов (Scene, NPCSpawner)	бит-в-бит: кэш инвалидируется при переприсвоении списка объектов
Точная часть подготовки полигонов без idiv	TexturingPers.paint9Pers(): div255 в drawmode 6 (3 места), 14 градиентов ((delta<<12)/dy) через ceil(2^37/dy) с коррекцией перелёта	бит-в-бит: div255 - тождество при |x| < 65535 (вне - прежний idiv), градиенты точны для всех int при dy >= 64 (вне - прежний idiv); перебор в docs/verify/test_setup_math.py

Что сделано	Где	Точность
FastMath.div50() и погода в int Q8	WeatherGenerator: координаты частиц в 1/256 пиксела вместо float; деление на 50 и на глубину 1..65 (INV_DEPTH) одним умножением; mixColor через div100	субпиксельно: div50 бит-в-бит на всём int, дробные константы равны прежним точно (fractions.Fraction), квантование 1/256 пиксела; TestWeather - инварианты 400 кадров
Развёртки циклов растеризатора	line_0_opaque (6x), paint9Mini (4x), Sprite.renderF (4x), fastbloomScr (4x), clearDisplay (8x)	бит-в-бит: быстрые пути включаются только для непрозрачных пикселов (RawImage.opaque)
Мелкие hoist-ы и кэши полей	MathUtils.pLength() (>> 2 вместо /4), Sprite.project() (kx/ky вместо четырёх чтений textures[0].rImg.scale), DirectX7.fDist255, SphereCast.edgeFar, Dijkstra.Point.dis	бит-в-бит; деление в pLength() оставлено на случай переполнения int

7. Готовые наборы

Каждый набор можно вставить в конец setting.txt (до первой строки с [группа], если она есть). Значения читаются при старте, поэтому после правки файл надо перезапустить MIDlet.

A. Побитово как оригинал (для сравнения скриншотов и отладки)

```
CPU_PROFILE=FAST
FAST_LIGHT=0
PERSPECTIVE_STEP=0
NEAR_ONLY=0
AI_LOD=0
MORPH_LOD=0
FAST_CULL=0
QCT=0
FAST_SETUP=0
RENDER_SCALE=100
NOKIA_BLIT=0
```

Все ускорения с потерей качества выключены, остаются только точные замены (div100, div50, isqrt, FrameClock, градиенты без idiv, развёртки циклов и т. п.). AI_LOD_DIST и MORPH_LOD_DIST при выключенных AI_LOD и MORPH_LOD ни на что не влияют; QCT=0 запрещает подхватывать упакованные .qct, если они есть в JAR; FAST_SETUP=0 возвращает прежние коэффициенты тумана; RENDER_SCALE=100 - прежнее разрешение рендера; NOKIA_BLIT=0 - прежний вывод кадра через g.drawRGB даже на Nokia. Погода хранит координаты в int Q8 (1/256 пиксела) без ключа - единственное субпиксельное отличие набора A.

B. Ничего не менять (рекомендуется)

```
; файл setting.txt оригинала - без единого нового ключа
```

На аппарате класса ARM9 автоопределение включит все щадящие значения по умолчанию, на быстром устройстве или в эмуляторе поведение останется прежним. Именно поэтому файлы оригинала продолжают работать как раньше.

C. Максимум скорости при умеренных потерях

```
CPU_PROFILE=ARM9
FAST_LIGHT=1
PERSPECTIVE_STEP=48
NEAR_ONLY=1
AI_LOD=1
AI_LOD_DIST=8000
MORPH_LOD=1
MORPH_LOD_DIST=9000
FAST_CULL=1
FAST_SETUP=1
RENDER_SCALE=75
```

Пороги LOD уменьшены, поэтому в редкий режим переходит больше объектов. RENDER_SCALE=75 оставляет 56 % пикселов 3D-сцены - HUD и прицел при этом остаются в родном разрешении. Профиль задан явно - настройки применяются даже там, где автоопределение не узнало аппарат.

D. Максимум скорости, потери заметны

```
CPU_PROFILE=ARM9
FAST_LIGHT=1
PERSPECTIVE_STEP=96
NEAR_ONLY=1
AI_LOD=1
AI_LOD_DIST=5000
MORPH_LOD=1
MORPH_LOD_DIST=6000
FAST_CULL=1
FAST_SETUP=1
RENDER_SCALE=50
```

RENDER_SCALE=50 оставляет 25 % пикселов 3D-сцены - самый сильный рычаг при нехватке fillrate; дополнительно можно снизить displaySize в меню настроек (он уменьшает уже весь кадр, включая HUD). Постэффекты от профиля не зависят: ими управляют родные ключи игры BLOOM, VIGNETTE в level.ini/setting.txt (DOUBLE_BRIGHT вырезан).

8. Сводка: что именно теряется при ухудшениях

Ключ	Видимый эффект
FAST_LIGHT=1	Освещение отличается на 0-4 единицы из 255 (в реальной сцене 0-1). Заметно только при покадровом сравнении.
PERSPECTIVE_STEP=48...96	Текстура на длинных наклонных сегментах (пол, потолок) слегка «плывёт» - коррекция выполняется реже.
NEAR_ONLY=1	Дальние объекты не обновляются: их ИИ и физика «застывают» до приближения игрока.
AI_LOD=1	Дальний NPC замечает игрока и перестраивает маршрут на один такт реакции позже.
MORPH_LOD=1	У дальних персонажей анимация переключается целыми кадрами, без плавного смешивания.
FAST_CULL=1	Ничего: отбрасывается только заведомо невидимое (подтверждено перебором 182 187 случаев).
FAST_SETUP=1	Коэффициенты тумана/подводной дымки могут отличаться на 1 из 255 (94 % совпадают точно). Заметно только при покадровом сравнении.
RENDER_SCALE=50...75	3D-сцена рисуется в меньшем разрешении и растягивается nearest-ом: картинка «крупнее зерном». HUD, прицел и тексты остаются чёткими.
NOKIA_BLIT=1	Ничего: drawPixels выводит пиксел-в-пиксел то же, что drawRGB (та же адресация и смысл transparency). На не-Nokia аппаратах автоматический откат; при сомнениях - NOKIA_BLIT=0.

Порядок включения по убыванию выгоды: сначала CPU_PROFILE=ARM9 - он включает все щадящие ветки разом, плюс снизьте displaySize в меню настроек (меньше пикселов в кадре - главный выигрыш), затем FAST_LIGHT и FAST_CULL, затем NEAR_ONLY и AI_LOD (они убирают работу по NPC - именно из-за них падал FPS в сценах с большим числом персонажей), и только потом PERSPECTIVE_STEP и MORPH_LOD. Постэффекты настраиваются отдельно родными ключами игры BLOOM/VIGNETTE.

9. Диагностика и проверка

При старте движок печатает итоговый профиль в `System.out` - в эмуляторе (KEmulator, FreeJ2ME, SquirrelJME) это видно в консоли/логе:

```
DeviceProfile: cpu=ARM9 platform=SonyEricssonK750 fastLight=1 nearOnly=1 aiLod=1 morphLod=1
fastCull=1 persStep=32
```

Как сравнивать и чем проверять

- Счётчик кадров: пункт «Show QFPS» в настройках (code.utils.QFPS вызывает FPS.frame() из модуля UtilsFPS). Сравнивать надо одну и ту же сцену - желательно NPC-тяжёлую, потому что именно там был провал.
- Контрольный замер: прогон с набором A (побитово как оригинал) против набора C или D. Разница - это вклад именно ключей; точные ускорения из раздела 6 действуют в обоих прогонах.
- Если FPS не растёт, первым делом проверьте строку DeviceProfile: при cpu=AUTO все щадящие значения по умолчанию выключены, и тогда ключи надо прописать явно (набор C).
- Проверки эквивалентности в репозитории (чистый Python, без телефона): docs/verify/test_fastmath_java.py (0 расхождений на 6 312 931 значении), test_light.py (отклонение 3 и 4 из 255), test_room_div.py (3020 пар, 0), test_sort.py (65 случаев + 50 кадров, 0; устойчивость равных - фикс мерцания §20), test_cull.py (182 187 случаев, 0 ошибочных отсечений), test_qct.py (round-trip и загрузчик движка побитово), test_setup_math.py (div255/div50/градиенты - 0 расхождений, туман +/-1 из 255, погода Q8 - инварианты 400 кадров), test_rms_settings.py (симметрия чтения/записи настроек RMS - регрессия на вылет второго запуска), test_glass.py (стекло режима 7: таблицы преломления бит-в-бит равны умножениям, 120 864 случая), test_modes913.py (div3/div255 бит-в-бит, FPq субпиксельно - 540 тыс. значений), test_ai_physics.py (физика/AI: кэш AABB и поднятые фильтры бит-в-бит - 360 тыс. сравнений), test_bbox.py (BoundingBox reSort/isVisible: 1620 кадров, 0 расхождений с оригиналом; свидетель - старый код давал 335 флипов isVisible), test_wz_div.py (деления /wz: 300 612 пар - A/fallback бит-в-бит, B <= 64 fp; 120 пролётов - отклонение <= 59 fp).
- Сборка JAR и проверка препроверки: docs/j2me-build/README.md, docs/j2me-build/build-j2me.sh, docs/j2me-build/check-jar.py (736 атрибутов StackMap, 0 StackMapTable, version 48).
- Полное описание всех изменений: docs/ARM9.md (разделы 3.2, 7, 9.1-9.9, 11-12).

Итог по совместимости. Ни один формат файла не изменён, ни один существующий ключ не переименован и не удалён, новые ключи опциональны и читаются из того же setting.txt. Оригинальный файл настроек продолжает работать; чтобы вернуть поведение оригинала побитово, достаточно набора A из раздела 7.